

Java Exception Handling

10 Real-Application Programs — Full Presentation Format

Name: S.Trivikram

Roll Number: A25126552251

Note: The on choices.Ar

Java Exception Handling: ArrayIndexOutOfBoundsException

Program: StudentMarks

1. Aim

To demonstrate ArrayIndexOutOfBoundsException handling using try-catch in a real-world student marks system.

2. Objective

To understand ArrayIndexOutOfBoundsException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a student marks system that performs an operation capable of causing ArrayIndexOutOfBoundsException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

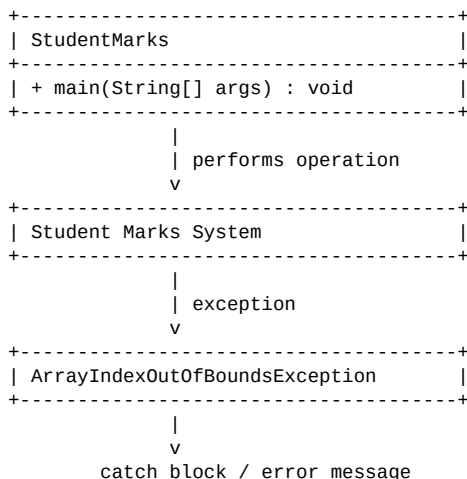
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required student marks system data or object is created.
3. The operation that may cause ArrayIndexOutOfBoundsException is placed inside the try block.
4. Java detects the invalid condition and ArrayIndexOutOfBoundsException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required student marks system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If ArrayIndexOutOfBoundsException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class StudentMarks {
```

```

public static void main(String[] args) {
    int[] marks = {80, 75, 90};
    try {
        System.out.println("Student Mark = " + marks[5]);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Error: Invalid mark position.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the student marks system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

ArrayIndexOutOfBoundsException — marks[5] is outside the valid array indexes, so ArrayIndexOutOfBoundsException is caught.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Invalid mark position.
```

10. Result

Thus, the Java program successfully demonstrates ArrayIndexOutOfBoundsException and handles the exception using try-catch.

Java Exception Handling: NullPointerException

Program: Customer

1. Aim

To demonstrate NullPointerException handling using try-catch in a real-world customer details system.

2. Objective

To understand NullPointerException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a customer details system that performs an operation capable of causing NullPointerException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

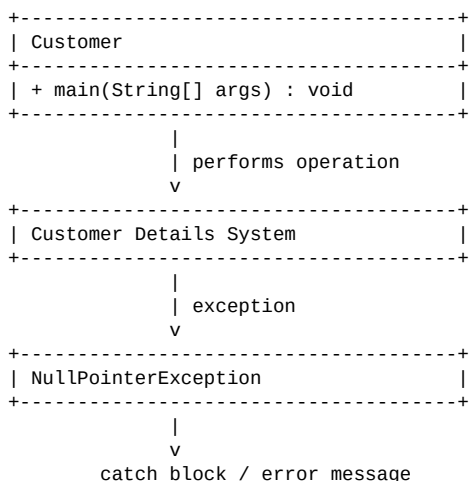
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required customer details system data or object is created.
3. The operation that may cause NullPointerException is placed inside the try block.
4. Java detects the invalid condition and NullPointerException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required customer details system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If NullPointerException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class Customer {
    public static void main(String[] args) {
```

```

String customerName = null;
try {
    System.out.println("Customer Name Length = "
        + customerName.length());
} catch (NullPointerException e) {
    System.out.println("Error: Customer name is not available.");
}
}

```

8. Code Explanation

Class declaration — Defines the class containing the customer details system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

NullPointerException — customerName is null, so calling length() causes NullPointerException.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Customer name is not available.
```

10. Result

Thus, the Java program successfully demonstrates NullPointerException and handles the exception using try-catch.

Java Exception Handling: NumberFormatException

Program: BankAccount

1. Aim

To demonstrate NumberFormatException handling using try-catch in a real-world bank account system.

2. Objective

To understand NumberFormatException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a bank account system that performs an operation capable of causing NumberFormatException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

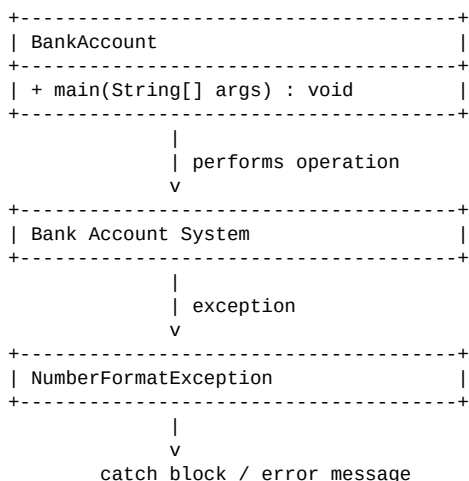
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required bank account system data or object is created.
3. The operation that may cause NumberFormatException is placed inside the try block.
4. Java detects the invalid condition and NumberFormatException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required bank account system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If NumberFormatException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class BankAccount {
    public static void main(String[] args) {
```

```

    String balance = "ABC";
    try {
        int amount = Integer.parseInt(balance);
        System.out.println("Balance = " + amount);
    } catch (NumberFormatException e) {
        System.out.println("Error: Invalid balance format.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the bank account system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

NumberFormatException — The string "ABC" is not a valid integer, so Integer.parseInt() causes NumberFormatException.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Invalid balance format.
```

10. Result

Thus, the Java program successfully demonstrates NumberFormatException and handles the exception using try-catch.

Java Exception Handling: StringIndexOutOfBoundsException

Program: MobileNumber

1. Aim

To demonstrate StringIndexOutOfBoundsException handling using try-catch in a real-world mobile number processing.

2. Objective

To understand StringIndexOutOfBoundsException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a mobile number processing that performs an operation capable of causing StringIndexOutOfBoundsException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

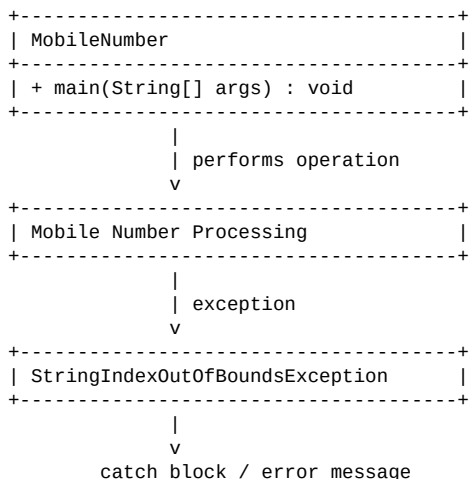
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required mobile number processing data or object is created.
3. The operation that may cause StringIndexOutOfBoundsException is placed inside the try block.
4. Java detects the invalid condition and StringIndexOutOfBoundsException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required mobile number processing data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If StringIndexOutOfBoundsException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class MobileNumber {
```



```

public static void main(String[] args) {
    String mobile = "9876543210";
    try {
        System.out.println("Digit = " + mobile.charAt(15));
    } catch (StringIndexOutOfBoundsException e) {
        System.out.println("Error: Invalid mobile number position.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the mobile number processing program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

StringIndexOutOfBoundsException — charAt(15) attempts to access a position outside the valid range of the mobile string.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Invalid mobile number position.
```

10. Result

Thus, the Java program successfully demonstrates StringIndexOutOfBoundsException and handles the exception using try-catch.

Java Exception Handling: ClassCastException

Program: EmployeeSystem

1. Aim

To demonstrate ClassCastException handling using try-catch in a real-world employee management system.

2. Objective

To understand ClassCastException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a employee management system that performs an operation capable of causing ClassCastException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

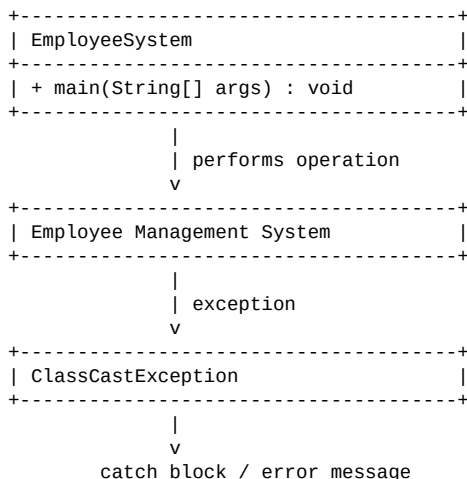
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required employee management system data or object is created.
3. The operation that may cause ClassCastException is placed inside the try block.
4. Java detects the invalid condition and ClassCastException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required employee management system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If ClassCastException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class Employee { }
```

```

class Manager extends Employee { }
class Developer extends Employee { }

class EmployeeSystem {
    public static void main(String[] args) {
        Employee emp = new Developer();
        try {
            Manager manager = (Manager) emp;
            System.out.println("Employee is a Manager.");
        } catch (ClassCastException e) {
            System.out.println("Error: Employee is not a Manager.");
        }
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the employee management system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

ClassCastException — The object stored in emp is a Developer, so casting it to Manager causes ClassCastException.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Employee is not a Manager.
```

10. Result

Thus, the Java program successfully demonstrates ClassCastException and handles the exception using try-catch.

Java Exception Handling: IllegalArgumentException

Program: DrivingLicense

1. Aim

To demonstrate IllegalArgumentException handling using try-catch in a real-world driving license validation.

2. Objective

To understand IllegalArgumentException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a driving license validation that performs an operation capable of causing IllegalArgumentException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

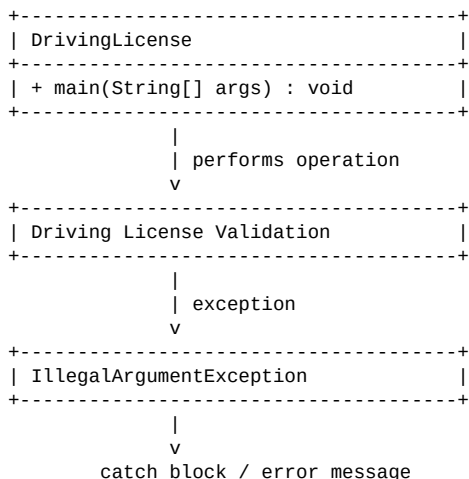
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required driving license validation data or object is created.
3. The operation that may cause IllegalArgumentException is placed inside the try block.
4. Java detects the invalid condition and IllegalArgumentException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required driving license validation data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IllegalArgumentException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class DrivingLicense {
```

```

static void checkAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException(
            "Age must be 18 or above.");
    }
    System.out.println("Eligible for driving license.");
}

public static void main(String[] args) {
    try {
        checkAge(16);
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

8. Code Explanation

Class declaration — Defines the class containing the driving license validation program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

IllegalArgumentException — checkAge(16) receives an invalid age and explicitly throws IllegalArgumentException.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Age must be 18 or above.
```

10. Result

Thus, the Java program successfully demonstrates IllegalArgumentException and handles the exception using try-catch.

Java Exception Handling: FileNotFoundException

Program: BankStatement

1. Aim

To demonstrate FileNotFoundException handling using try-catch in a real-world bank statement system.

2. Objective

To understand FileNotFoundException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a bank statement system that performs an operation capable of causing FileNotFoundException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

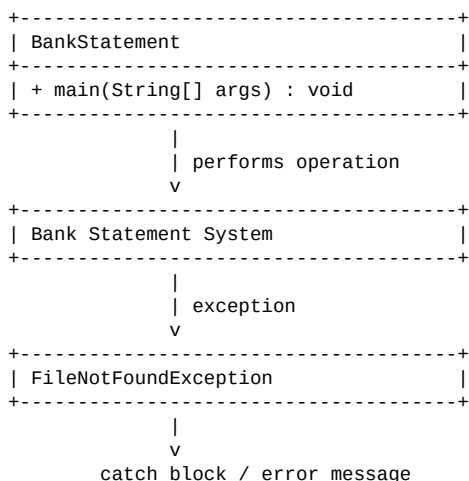
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required bank statement system data or object is created.
3. The operation that may cause FileNotFoundException is placed inside the try block.
4. Java detects the invalid condition and FileNotFoundException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required bank statement system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If FileNotFoundException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
import java.io.File;
import java.io.FileNotFoundException;
```

```
import java.util.Scanner;

class BankStatement {
    public static void main(String[] args) {
        try {
            File file = new File("bankstatement.txt");
            Scanner sc = new Scanner(file);
            System.out.println("Bank statement opened successfully.");
            sc.close();
        } catch (FileNotFoundException e) {
            System.out.println("Error: Bank statement file not found.");
        }
    }
}
```

8. Code Explanation

Class declaration — Defines the class containing the bank statement system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

FileNotFoundException — The program attempts to open bankstatement.txt. If it is unavailable, FileNotFoundException is caught.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Bank statement file not found.
```

10. Result

Thus, the Java program successfully demonstrates FileNotFoundException and handles the exception using try-catch.

Java Exception Handling: IOException

Program: EmployeeFile

1. Aim

To demonstrate IOException handling using try-catch in a real-world employee file system.

2. Objective

To understand IOException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a employee file system that performs an operation capable of causing IOException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

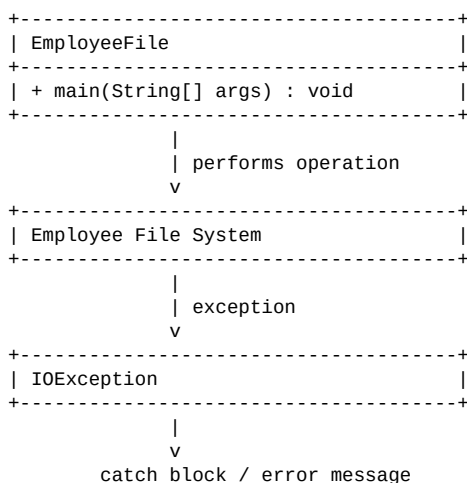
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required employee file system data or object is created.
3. The operation that may cause IOException is placed inside the try block.
4. Java detects the invalid condition and IOException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required employee file system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IOException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
import java.io.FileReader;
import java.io.IOException;

class EmployeeFile {
```



```

public static void main(String[] args) {
    try {
        FileReader file = new FileReader("employee.txt");
        int data = file.read();
        System.out.println("Employee data read successfully.");
        file.close();
    } catch (IOException e) {
        System.out.println("Error: Unable to read employee file.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the employee file system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

IOException — `FileReader/read()` performs file input that may produce an `IOException`, which is handled by `catch`.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Unable to read employee file.
```

10. Result

Thus, the Java program successfully demonstrates `IOException` and handles the exception using `try-catch`.

Java Exception Handling: SecurityException

Program: CompanySecurity

1. Aim

To demonstrate SecurityException handling using try-catch in a real-world company security system.

2. Objective

To understand SecurityException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a company security system that performs an operation capable of causing SecurityException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

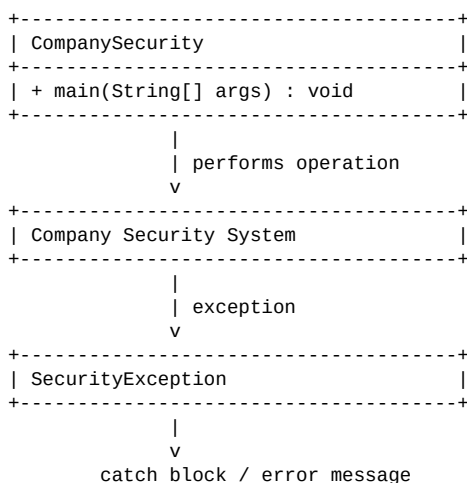
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required company security system data or object is created.
3. The operation that may cause SecurityException is placed inside the try block.
4. Java detects the invalid condition and SecurityException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required company security system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If SecurityException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class CompanySecurity {
    public static void main(String[] args) {
        try {
            throw new SecurityException(
```

```

        "Access denied to company resource.");
    } catch (SecurityException e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the company security system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

SecurityException — The program deliberately throws SecurityException to demonstrate how a security restriction can be handled.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Access denied to company resource.
```

10. Result

Thus, the Java program successfully demonstrates SecurityException and handles the exception using try-catch.

Java Exception Handling: IllegalStateException

Program: OnlineOrder

1. Aim

To demonstrate IllegalStateException handling using try-catch in a real-world online shopping order system.

2. Objective

To understand IllegalStateException, identify the operation that causes it, and handle it safely using the try-catch mechanism.

3. Task Description

Create a Java program for a online shopping order system that performs an operation capable of causing IllegalStateException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

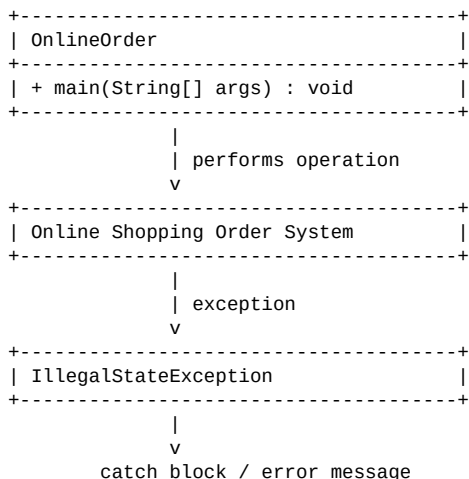
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required online shopping order system data or object is created.
3. The operation that may cause IllegalStateException is placed inside the try block.
4. Java detects the invalid condition and IllegalStateException occurs.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required online shopping order system data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IllegalStateException occurs, transfer control to the catch block.
6. Display an appropriate exception message.
7. Stop.

6. UML Class Diagram



7. Program Code

```
class OnlineOrder {
```

```

static boolean orderPlaced = false;

static void cancelOrder() {
    if (!orderPlaced) {
        throw new IllegalStateException(
            "Order cannot be cancelled before placing.");
    }
    System.out.println("Order cancelled successfully.");
}

public static void main(String[] args) {
    try {
        cancelOrder();
    } catch (IllegalStateException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

8. Code Explanation

Class declaration — Defines the class containing the online shopping order system program.

main() — The starting point of program execution.

try — Contains the operation that may generate an exception.

IllegalStateException — The order is not placed, so attempting to cancel it causes `IllegalStateException`.

catch — Catches the exception and prevents abnormal termination.

System.out.println() — Displays a user-friendly message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Error: Order cannot be cancelled before placing.
```

10. Result

Thus, the Java program successfully demonstrates `IllegalStateException` and handles the exception using try-catch.

Summary of 10 Exceptions

No.	Exception	Real Application
1	ArrayIndexOutOfBoundsException	Student Marks System
2	NullPointerException	Customer Details System
3	NumberFormatException	Bank Account System
4	StringIndexOutOfBoundsException	Mobile Number Processing
5	ClassCastException	Employee Management System
6	IllegalArgumentException	Driving License Validation
7	FileNotFoundException	Bank Statement System
8	IOException	Employee File System
9	SecurityException	Company Security System
10	IllegalStateException	Online Shopping Order System